Laporan Praktikum 3

Mata Kuliah Teknologi IoT II

Oleh :

Nama : Sofy Salsabila
NPM : 25782084
Kelas : TRI – 3C

Dosen Pengampu :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T., IPM. ASEAN Eng.

Agus Ambarwari, S.Pd, M.Kom

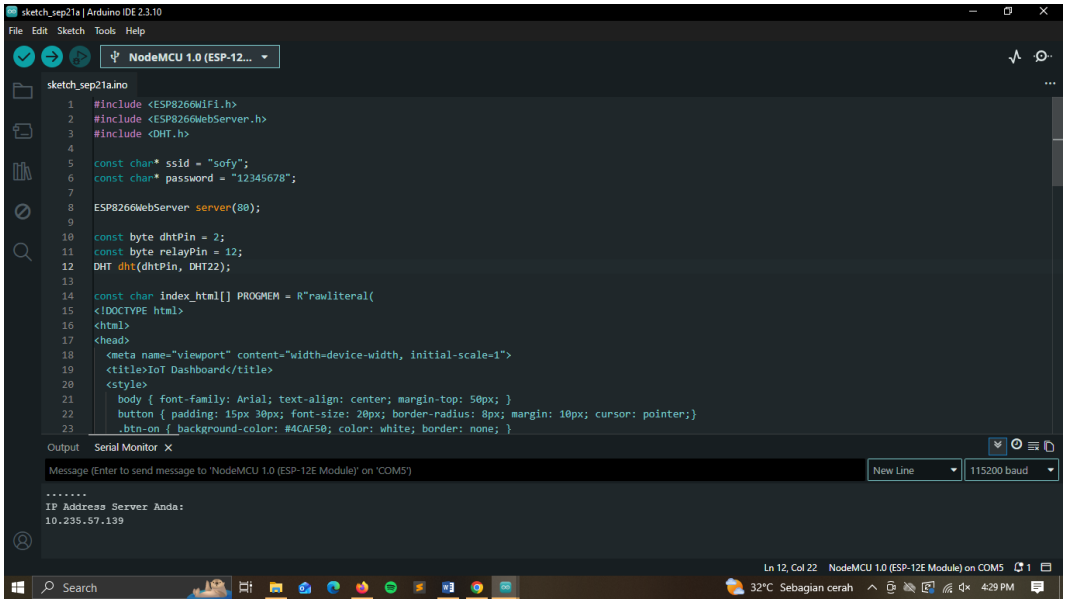
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2025/2026

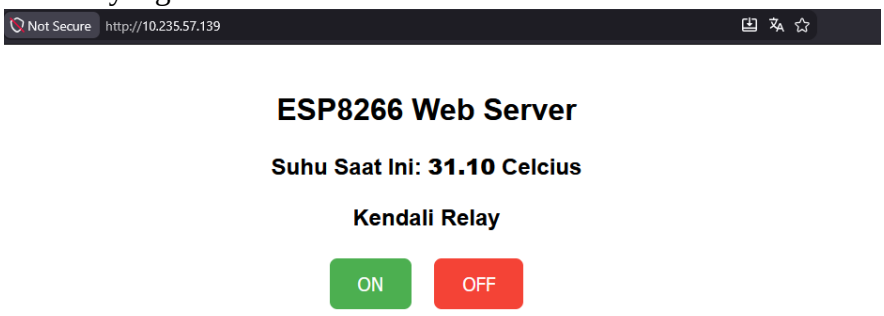
Modul 3: Jaringan Nirkabel dan Komunikasi HTTP

Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)

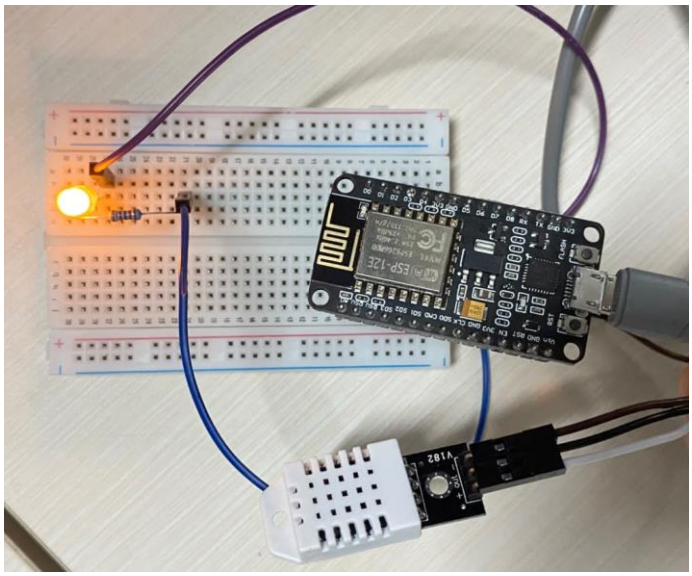
Ketika program dijalankan maka akan menampilkan ip address I serial monitor kita. Lalu membuka ip address di web untuk memantau monitoringnya lewat web server tersebut.



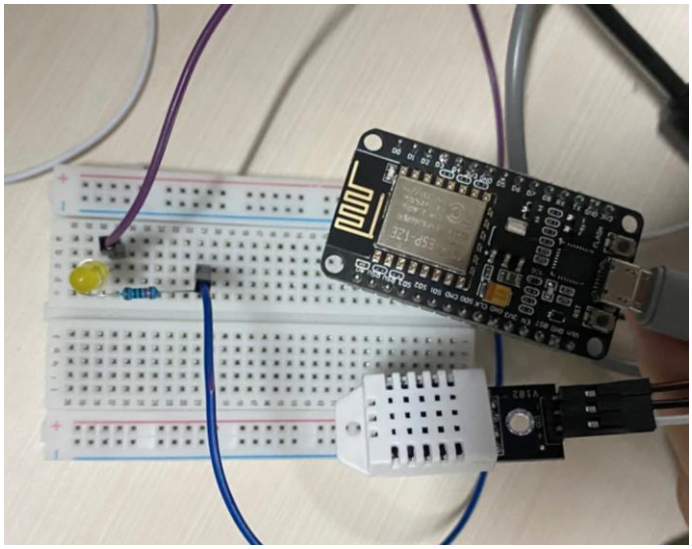
Web Server untuk mengendalikan LED nya, untuk membukanya menggunakan IP address yang dikirimkan di serial monitor lalu dibuka di browser.



Ketika di ON atau mengidupkan kan LED di Web akan hidup LED nya



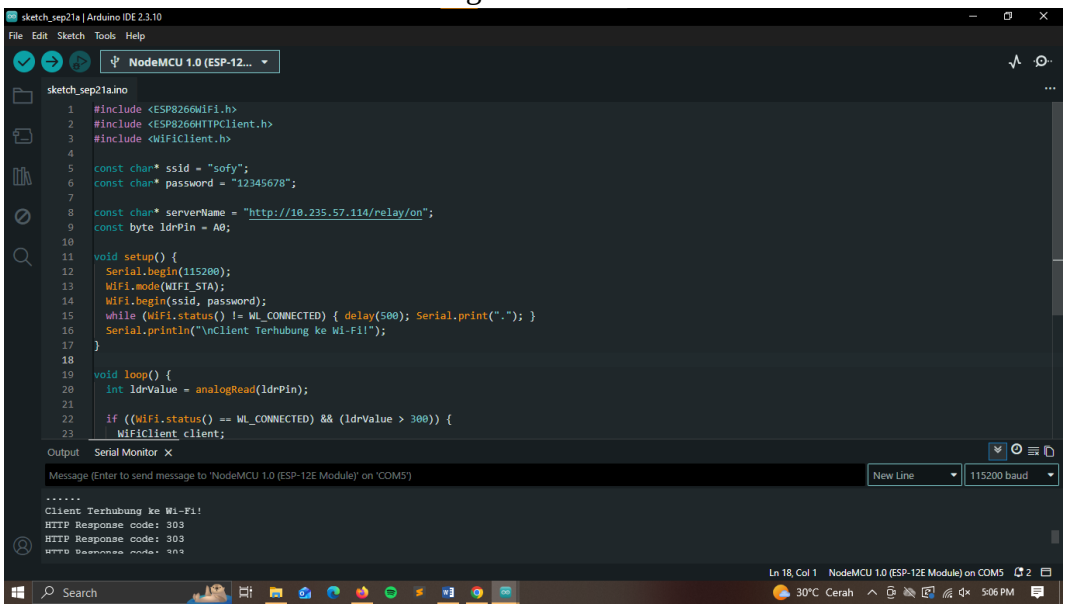
Ketika di OFF atau mematikan kan LED di Web akan mati LED nya



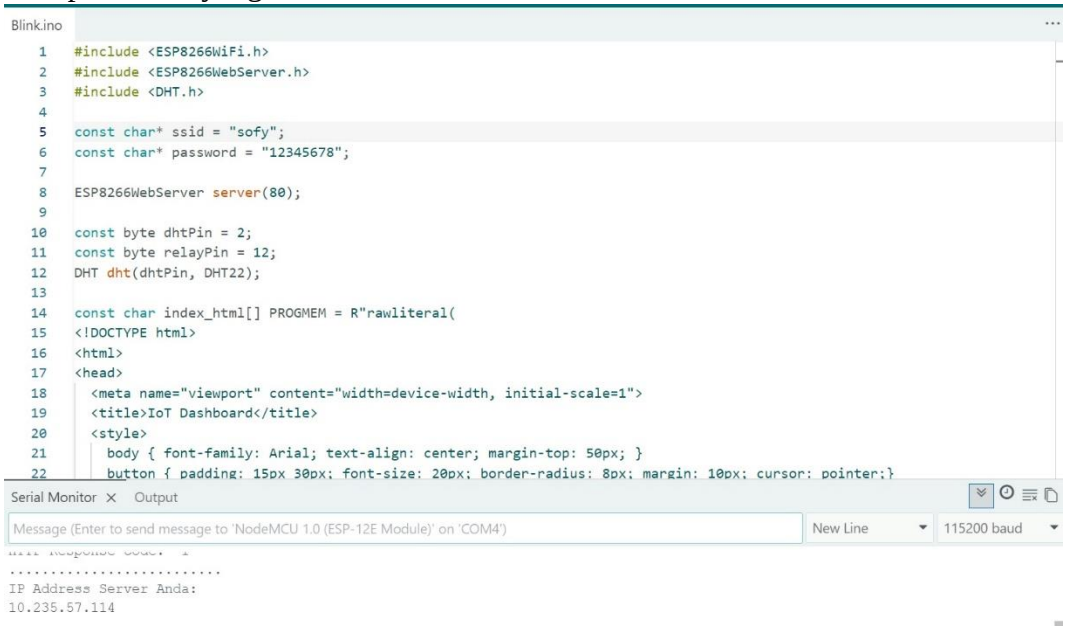
Jadi praktikum 1 ini, saya menggunakan NodeMCU sebagai Web Server yang terhubung ke Wi-Fi. Sensor DHT22 digunakan untuk membaca suhu, kemudian hasilnya ditampilkan di web. Selain itu, LED dapat dikendalikan melalui tombol **ON dan OFF** yang tersedia di halaman web. Setelah NodeMCU terhubung ke Wi-Fi, alamat IP ditampilkan pada Serial Monitor dan IP tersebut digunakan untuk membuka Web Server melalui browser. Hasil pengujian menunjukkan suhu berhasil tampil di web dan LED dapat dinyalakan maupun dimatikan melalui tombol di web**.**

Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)

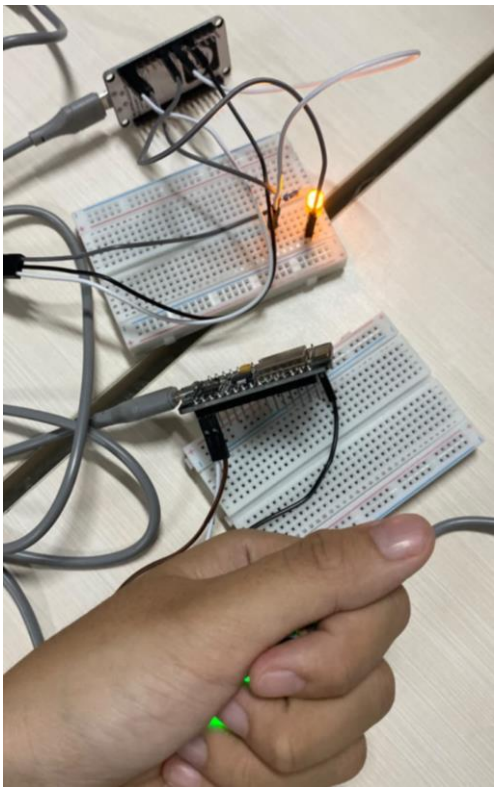
Punya saya sebagai client, dan ketika dijalankan dan terhubung, serial monitor memberitahu bahwa client terhubung ke server teman.



Punya teman sebagai server, menampilkan ip address setelah terhubung ke client dan ip tersebut yang berarti sudah connect.



Ketika LDR punya client di tutup dan LED server menjadi menyala



Jadi untuk praktikum 2 ini**,** NodeMCU digunakan sebagai HTTP Client yang membaca kondisi cahaya menggunakan sensor LDR. Ketika LDR ditutup hingga gelap dan nilai LDR kurang dari 300, Client mengirim HTTP GET Request ke IP NodeMCU Server untuk menjalankan /relay/on, sehingga LED pada Server menyala. Hasil pengujian pada Serial Monitor Client menunjukkan HTTP Response code: 303, yang berarti request berhasil. Praktikum ini tidak membuat tampilan web baru, karena fokusnya adalah komunikasi antara Client dan Server melalui HTTP.

Latihan

- **Latihan 1 (Pemahaman Jaringan):** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?
  Jawab : Menggunakan mode **Station (STA)** karena pada kedua praktikum NodeMCU perlu **terhubung ke jaringan Wi-Fi** agar dapat berkomunikasi dengan perangkat lain.
    - Pada **Praktikum 1**, NodeMCU berperan sebagai **Web Server** dan terhubung ke Wi-Fi untuk menerima akses dari HP/laptop melalui alamat IP.

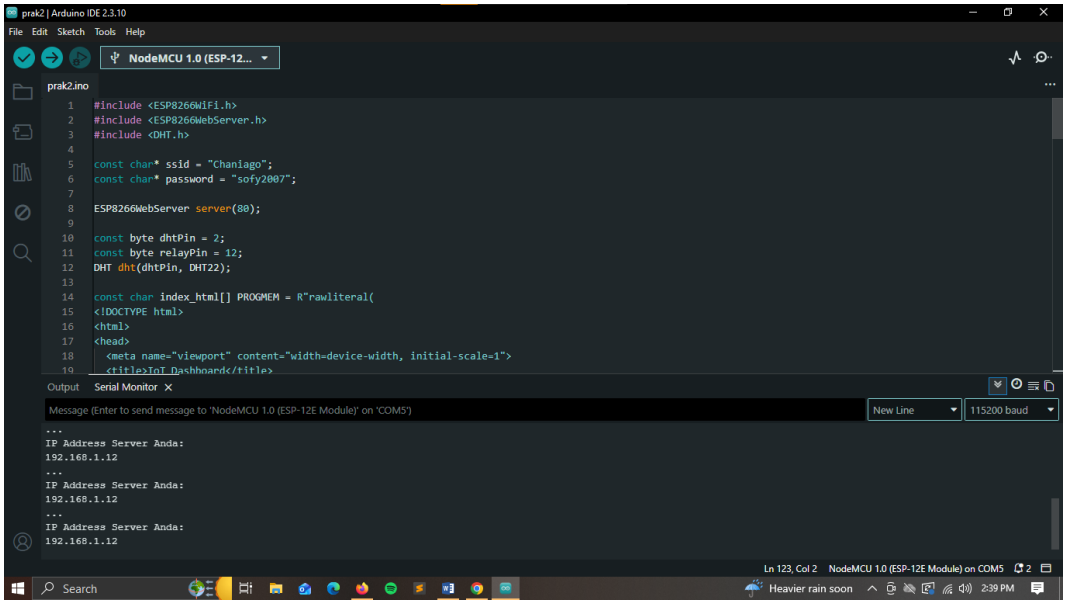
- Pada **Praktikum 2**, NodeMCU berperan sebagai **HTTP Client** dan harus terhubung ke Wi-Fi yang sama dengan Server agar dapat mengirim **HTTP GET** ke alamat IP Server.

Sedangkan mode **Access Point (AP)** membuat NodeMCU menjadi pemancar Wi-Fi sendiri. Mode tersebut tidak sesuai dengan skenario praktikum karena tujuan praktikum adalah membuat **Client dan Server berkomunikasi melalui jaringan Wi-Fi yang sama**.

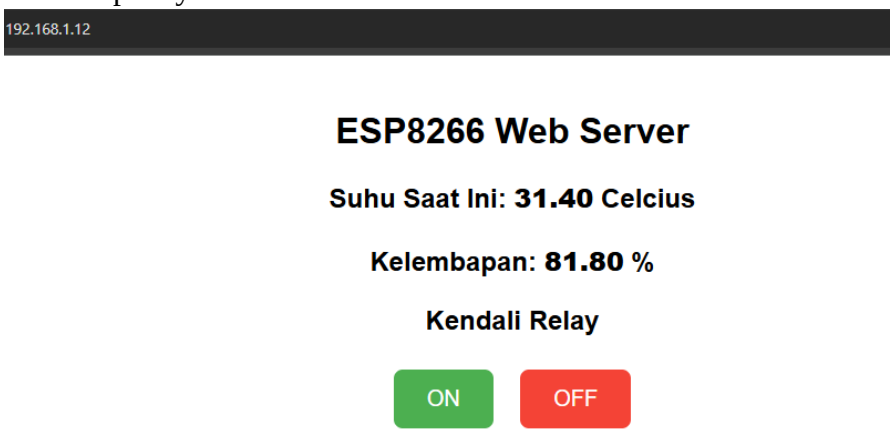
- **Latihan 2 (Modifikasi Antarmuka & Telemetri):** Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:
  i. **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman *web*.

  Jawab : Pada latihan ini, saya memodifikasi Web Server yang sebelumnya hanya menampilkan suhu agar dapat menampilkan suhu dan kelembapan dari sensor DHT. Saya menambahkan pembacaan kelembapan menggunakan dht.readHumidity(), kemudian hasilnya ditampilkan di web menggunakan placeholder %HUMIDITY%. Setelah saya jalankan diweb berhasil menampilkan suhu 31,40 °C dan kelembapan 81,80%**.** Nilai tersebut merupakan hasil pembacaan langsung dari sensor DHT dan akan berubah ketika halaman web dimuat atau di-refresh kembali. Lalu, pada LED sama saja seperti paraktikum 1, jika di ON kan akan menyala dan jika di OFF kan akan mati.

Setelah dijalankan akan serial monitor menampilkan ip addres yang akan dibuka di web untuk monitoring



Ketika ip address dibuka di web akan menampilkan suhu dan juga kelembapannya



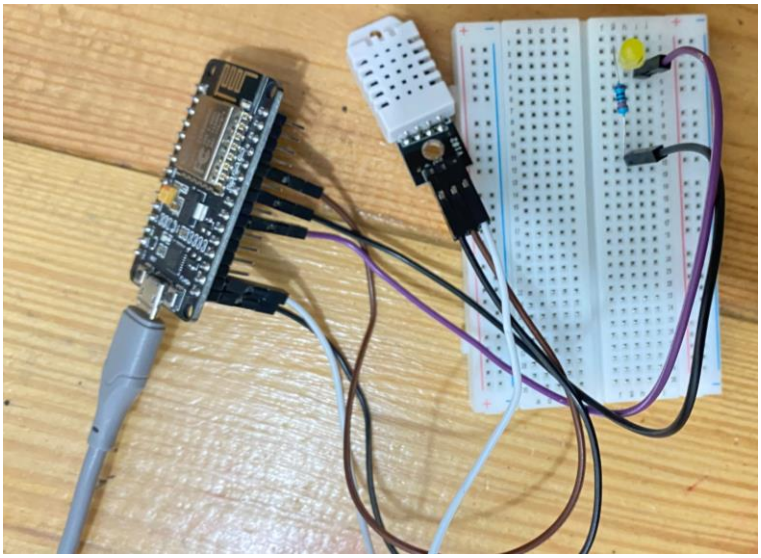
ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi **LED** saat ini:

- Jika LED sedang **OFF**, tombol menampilkan tulisan **"NYALAKAN (ON)"** (misal: warna hijau/biru) dan mengarahkan aksi ke `/relay/on`.

Disini menampilkan tombol **NYALAKAN (ON)** dan LED blm hidup karena belum di pencet tombolnya







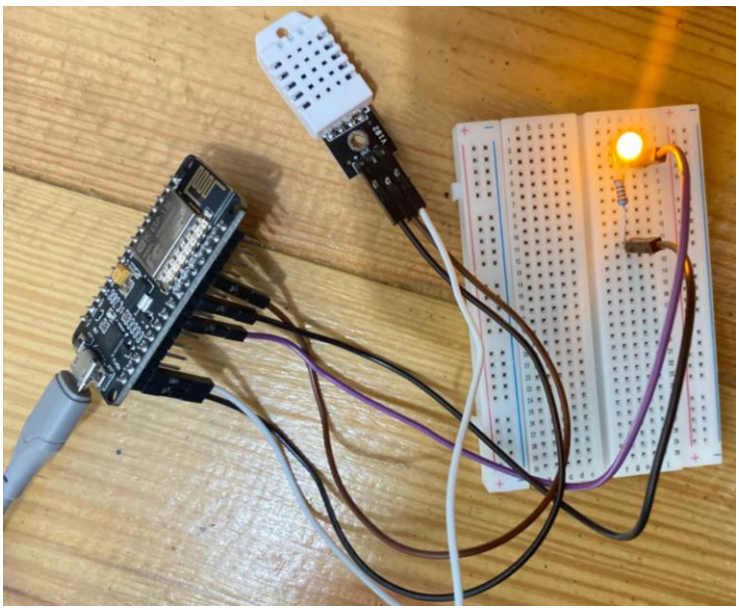
- Jika LED sedang **ON**, tombol otomatis berubah menjadi tulisan **"MATIKAN (OFF)"** (warna merah) dan mengarahkan aksi ke `/relay/off`. *(Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti* `%RELAY_BUTTON%`*, lalu di fungsi* `handleRoot()`*, periksa status pin relay/LED menggunakan* `digitalRead(relayPin)` *sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai).* Tuliskan potongan baris kode HTML dan fungsi `handleRoot()` yang Anda ubah!

  Jawab : Pada latihan ini, saya mengubah tampilan Web Server agar tombol ON dan OFF yang sebelumnya terpisah menjadi satu tombol saja. Tombol tersebut dapat berubah sesuai kondisi LED dengan mengecek status relayPin menggunakan digitalRead(relayPin). Jika LED dalam

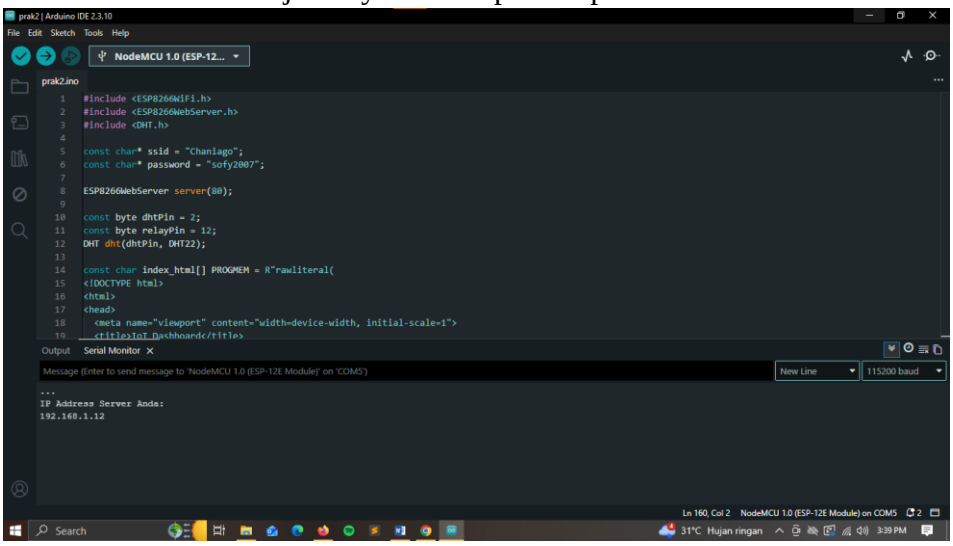
kondisi mati, maka pada halaman web muncul tombol **“NYALAKAN (ON)”** berwarna hijau dan ketika ditekan akan menjalankan /relay/on. Setelah LED menyala, tombol berubah menjadi **“MATIKAN (OFF)”** berwarna merah dan menjalankan /relay/off.

Disini menampilkan tombol **MATIKAN (OFF),** yang berarti tombol **NYALAKAN (ON)** sudah saya pencet dan LED hidup.



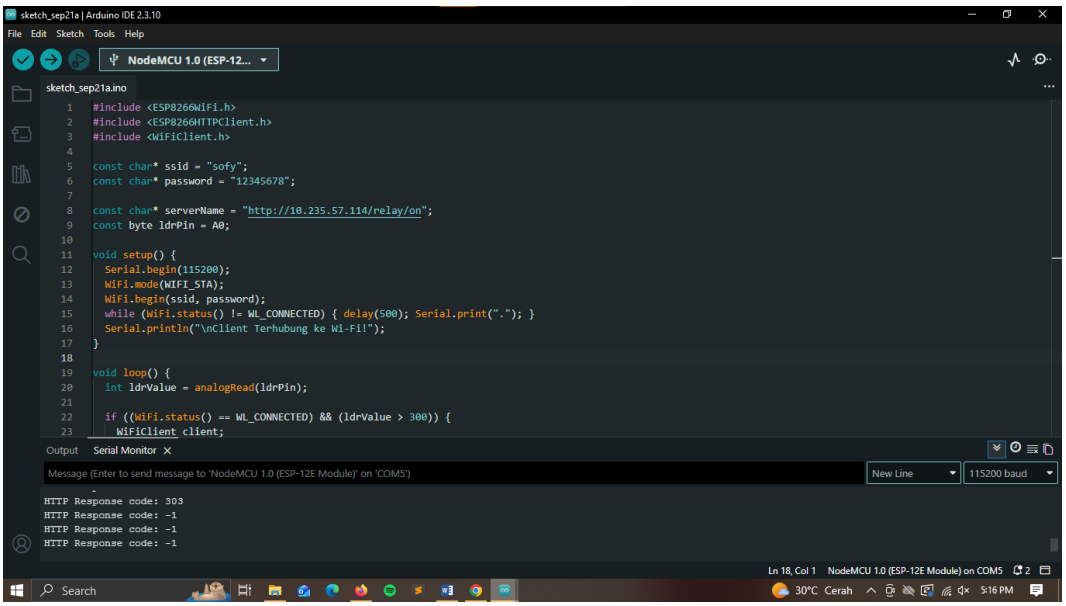


Serial monitor sama aja hanya menampilkan ip address



- **Latihan 3 (Analisis Kode Client):** Pada Praktikum 2, *Serial Monitor Client* mencetak `HTTP Response code`. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

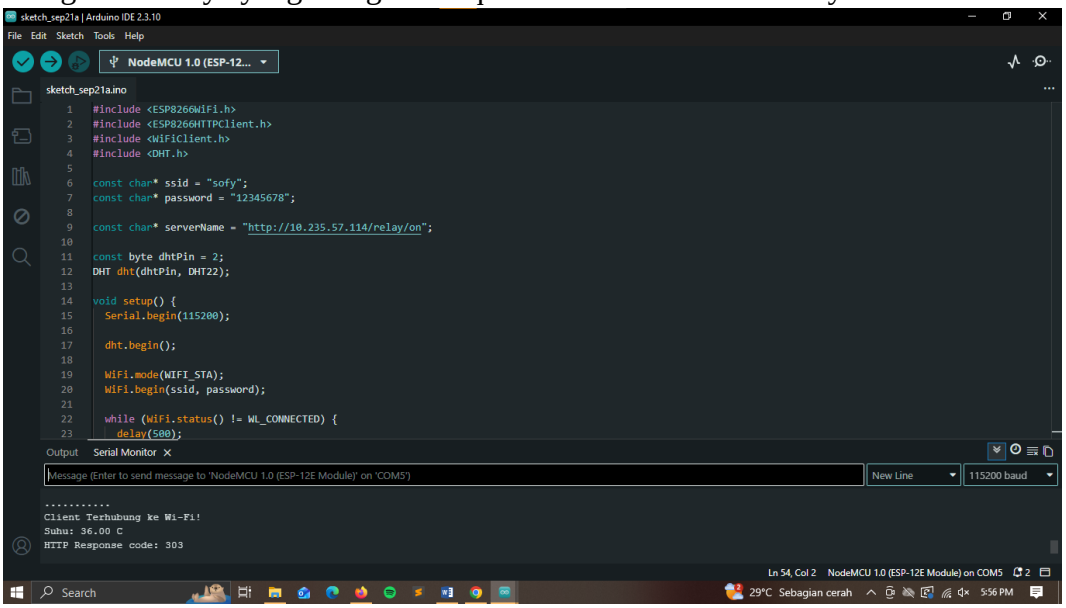
  Jawab : Dari yang tehubung dan bisa mengirim respons, dan ketika punya teman yang sebagai server dicabut akan menampilkan HTTP Response code: -1, yang berarti gagal terhubung.



- **Latihan 4 (Pengembangan Algoritma):** Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* `if` di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

  Jawab : Pada latihan ini, Client menggunakan sensor DHT22 untuk membaca suhu lingkungan. Program pada Client dimodifikasi sehingga HTTP GET Request hanya dikirim ketika suhu yang terbaca lebih dari 35°C. Saat pengujian, sensor membaca suhu 36,00°C, sehingga kondisi suhu > 35 terpenuhi. Client kemudian mengirim GET Request ke Server, dan Server memberikan respons dengan kode 303 yang terlihat pada Serial Monitor Client. Jadi, pengujian ini berfokus pada komunikasi Client–Server melalui HTTP, tidak di web di browser.

  sebagai client nya yang mengirim request dan server menerimanyaa.

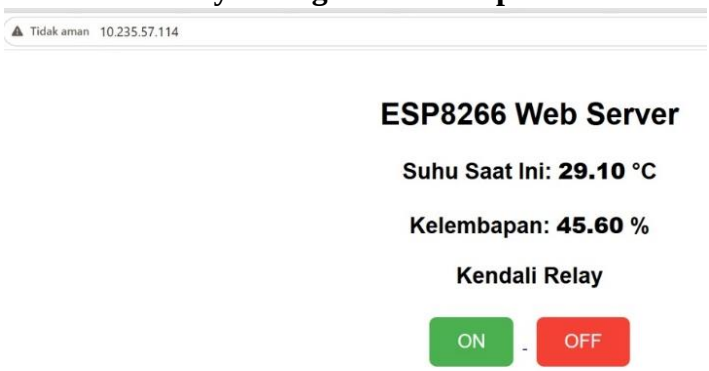


Tugas

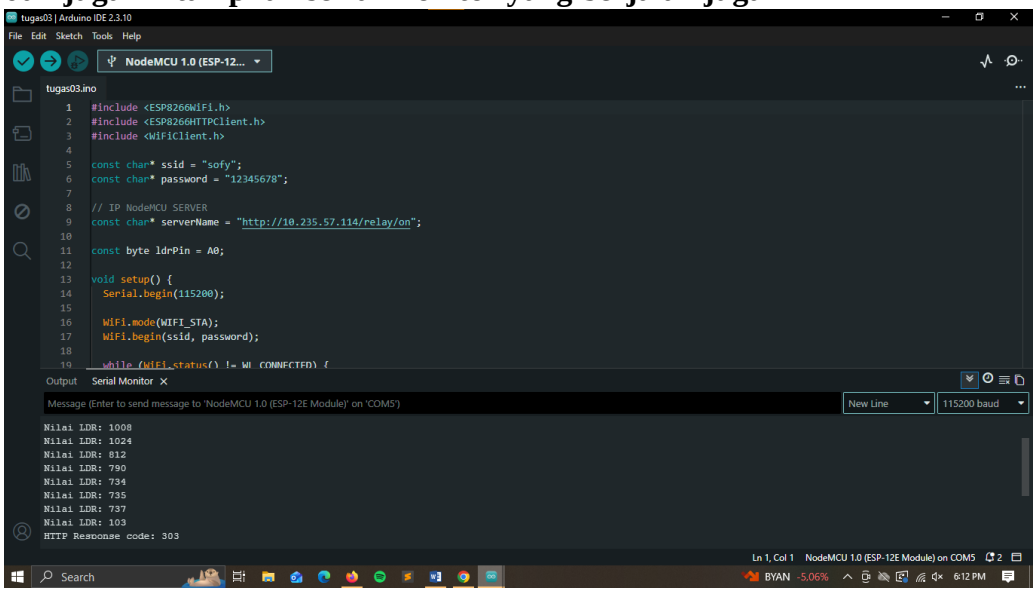
Pada tugas ini, kode NodeMCU Server ditambahkan <meta http-equiv="refresh" content="5"> di dalam bagian <head> pada index_html, sehingga browser otomatis memuat ulang halaman setiap 5 detik. Dengan begitu, nilai suhu dan kelembapan dari DHT22 pada Web Dashboard akan diperbarui secara otomatis tanpa menekan tombol refresh.

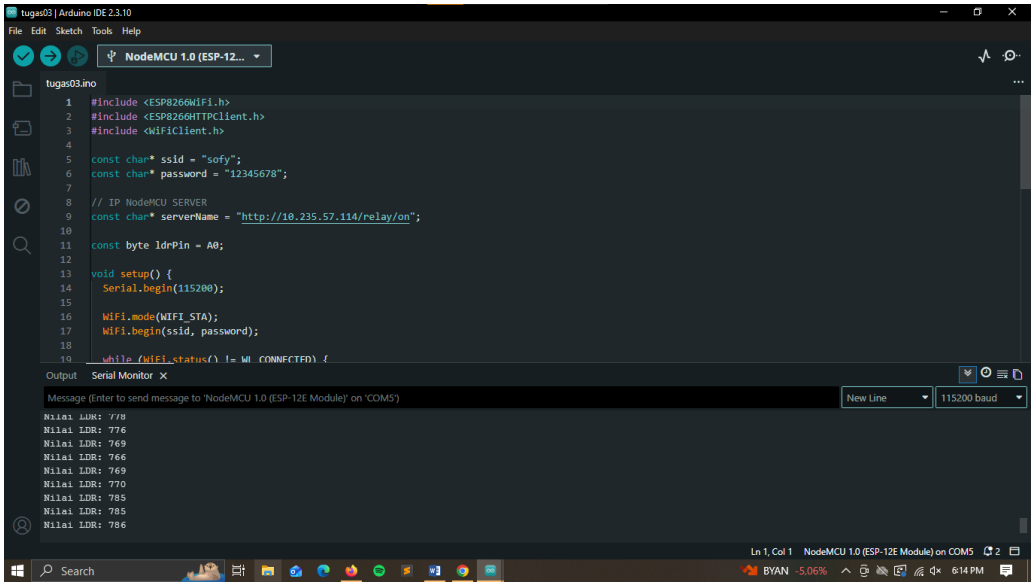
Sementara itu, Serial Monitor Client tetap menampilkan nilai LDR dan HTTP Response code seperti 303 ketika Client berhasil mengirim request ke Server. LED Server tetap mengikuti kondisi terakhirnya; jika ON tetap menyala dan jika OFF tetap mati. Jadi, yang otomatis refresh setiap 5 detik adalah halaman web Server, bukan Client atau LED.

ini web servernya mengrefresh setiap 5 detik.



dan juga ini tampilan serial monitor yang berjalan juga




tugas03 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
tugas03.ino
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
const char* ssid = "sofy";
const char* password = "12345678";
// IP NodeMCU SERVER
const char* serverName = "http://10.235.57.114/relay/on";
const byte ldrPin = A0;
void setup() {
Serial.begin(115200);
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')
New Line
115200 baud
Nilai LDR: 776
Nilai LDR: 769
Nilai LDR: 766
Nilai LDR: 769
Nilai LDR: 770
Nilai LDR: 785
Nilai LDR: 785
Nilai LDR: 786
Ln 1, Col 1 NodeMCU 1.0 (ESP-12E Module) on COM5
Search
BYAN -5.06%
6:14 PM

